Übungsblatt 2: Scheduling-Theorie und kooperativer Dispatcher

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Ziele, Voraussetzungen und Zeitplan

Sie beschreiben Jobs mit Zeitparametern, prüfen Fristen unter verschiedenen Regeln und erklären einen kooperativen Aufrufring. Vorausgesetzt werden Vorlesung 1 und die heutige Theorie: C-Funktionen, Variablen, Schleifen, Zeitvertrag und Interrupts. Ein Echtzeitbetriebssystem wird für den Programmvergleich nicht eingesetzt. Funktionszeiger und Verwaltungsstrukturen werden in der Vorlesung erklärt.

Abschnitt	Minuten	Umfang
Fünf Papieraufgaben mit Besprechung	50	32 Punkte
Vorbereiteter Programmvergleich	25	vier Varianten und Protokoll

Zusammen mit 90 Minuten Vortrag und 15 Minuten Quiz ergibt dies **180 Minuten**. Installation, erster Dual-Core-Start und erstmalige Moduleinbindung sind Vorbereitung. Die Papieraufgaben sind vollständig ohne Board lösbar. Die vollständigen Anwendungsbausteine und Integrationsschritte liegen für alle Studierenden in code/lab02.

Der Programmvergleich verwendet **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Die Anwendung läuft auf Cortex-M4 ohne Kernel; Cortex-M7 erhält den vorbereiteten generierten Start. Ein fertig erzeugtes Boardprojekt und ausgeführte Hardwaremessungen werden nicht behauptet. Theorie und Messwerte bleiben getrennt.

Teil A: Papieraufgaben

Gemeinsames Modell und Notation

Sofern nicht anders angegeben: genau ein Prozessor, unabhängige periodische Aufgaben, vollständig präemptierbare Jobs, keine Sperren oder Selbstsuspendierung, kein Freigabejitter. Interrupt-, Auswahl- und Wechselkosten sind null. Ausführungszeiten sind vorgegebene obere Schranken. Diese **Lehrannahmen** sind keine Boardmessung und gelten nicht automatisch für den kooperativen Code in Teil B.

Größe	Bedeutung
C_i	eigener Prozessorbedarf eines Jobs der Aufgabe τ_i
T_i	Abstand periodischer Freigaben
D_i	relative Abschlussfrist ab Freigabe
$r_{i,k}$, $s_{i,k}$, $f_{i,k}$	Freigabe, erster Start, Abschluss des Jobs k
$d_{i,k} = r_{i,k} + D_i$	absolute Deadline
$R_{i,k} = f_{i,k} - r_{i,k}$	Antwortzeit des einzelnen Jobs
R_i	nachgewiesene maximale Antwortzeit im Modell

Task bedeutet wiederkehrende Aufgabe, Job eine einzelne Aktivierung. Zeitintervalle sind halboffen: $[a, b)$ enthält den Anfang, aber nicht den Endpunkt. Abschluss genau auf der Deadline gilt als rechtzeitig.

Aufgabe 1: Ein Job auf der Zeitachse · 8 Minuten · 6 Punkte

Eine Aufgabe hat $T = 10\,\text{ms}$ und $D = 8\,\text{ms}$. Ein Job wird bei $r = 0$ freigegeben. Er wartet bis $s = 2$, rechnet von 2 bis 3, wird von einer anderen Aufgabe verdrängt und rechnet von 5 bis 7 weiter. Alle Zeitpunkte sind in Millisekunden.

1. Zeichnen Sie die Belegung dieses Jobs in $[0, 10)$. Kennzeichnen Sie Freigabe, Start, beide Rechenstücke, Abschluss, absolute Deadline und nächste Freigabe. (3 P)
2. Berechnen Sie eigene Prozessorzeit, Startlatenz und Antwortzeit. Prüfen Sie die Frist. Warum sind beide Rechenstücke derselbe Job? (2 P)
3. Ist die verdrängte Aufgabe von 3 bis 5 Ready oder Blocked? Begründen Sie anhand ihrer Warteursache. (1 P)

Aufgabe 2: RMS-Schranke richtig interpretieren · 8 Minuten · 6 Punkte

Für alle drei Aufgaben gilt $D_i = T_i$:

Aufgabe	C_i in ms	T_i in ms
τ_1	1	4
τ_2	2	6
τ_3	2	12

1. Berechnen Sie $U = \sum_i C_i/T_i$ als Bruch und Prozentwert. (2 P)
2. Vergeben Sie Prioritäten nach RMS, Rate Monotonic Scheduling. Prüfen Sie die hinreichende Liu-und-Layland-Schranke $U \leq n(2^{1/n} - 1)$ für $n = 3$. (2 P)
3. Was würde ein Überschreiten dieser Schranke bei weiterhin $U \leq 1$ bedeuten? Darf der bestandene Test unverändert auf den kooperativen Round-Robin aus Teil B übertragen werden? Begründen Sie beide Antworten. (2 P)

Aufgabe 3: Antwortzeit trotz überschrittener Schranke · 14 Minuten · 8 Punkte

Dieses Set entspricht dem 85-%-Beispiel der Vorlesung; es ist **ein anderes Set als in Aufgabe 2**. Wieder gilt $D_i = T_i$:

Aufgabe	C_i in ms	T_i in ms
τ_1	1	4
τ_2	2	5
τ_3	2	10

Die RTA, Response-Time Analysis oder Antwortzeitanalyse, verwendet für feste Prioritäten:

$$R_i^{(0)} = C_i, \qquad R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j.$$

$hp(i)$ ist die Menge höher priorisierter Aufgaben, $\lceil x \rceil$ rundet auf die nächste ganze Zahl auf. Iterieren Sie bis zu einem unveränderten Wert oder einer Überschreitung der Deadline.

1. Bestimmen Sie U und die RMS-Reihenfolge. Welche Aussage liefert allein die Liu-und-Layland-Schranke? (2 P)
2. Berechnen Sie R_1, R_2 und alle Iterationswerte für R_3 mit Zwischenrechnungen. (4 P)
3. Prüfen Sie jede Frist und erklären Sie, warum $\lceil 5/5 \rceil = 1$ gilt. Darf aus dem Ergebnis „85 % ist unter RMS immer sicher“ folgen? (2 P)

Aufgabe 4: RMS und EDF am selben Set · 12 Minuten · 6 Punkte

Zwei Aufgaben starten synchron bei null:

Aufgabe	C_i in ms	$T_i = D_i$ in ms
τ_1	2	5
τ_2	4	7

EDF bedeutet Earliest Deadline First: Aus den bereiten Jobs wird der mit der frühesten **absoluten** Deadline gewählt. Bei Gleichstand bleibt der laufende Job. Es gilt das gemeinsame präemptive Modell.

1. Berechnen Sie U. Zeichnen Sie unter RMS die Belegung bis 7 ms und zeigen Sie die Fristverletzung. (2 P)
2. Zeichnen Sie unter EDF die Belegung bis 12 ms. Beschriften Sie die Jobs und begründen Sie die Entscheidungen bei 5 und 10 ms mit den absoluten Deadlines. (3 P)
3. Was bedeutet dieses Ergebnis für den Unterschied zwischen „unter RMS nicht einplanbar“ und „überhaupt nicht machbar“? Begründen Sie die EDF-Aussage für alle periodischen Jobs, nicht nur den gezeichneten Ausschnitt. (1 P)

Aufgabe 5: Kooperative Runden und Jitter · 8 Minuten · 6 Punkte

Hier gilt ausdrücklich ein **anderes Modell**: Drei Schritte A, B und C sind ständig bereit. Der Dispatcher ruft sie zyklisch A, B, C auf. Jeder Schritt kehrt nach seiner Arbeit zurück. Es gibt keine erzwungenen Zeitscheiben und keine periodischen Jobfreigaben. Erster A-Start ist bei null; Aufruf- und Interruptkosten sind null.

A benötigt 1 ms, C ebenfalls 1 ms. Ein Marker ist während der Arbeit von A High. Definieren Sie Pulsbreite $w_k = f_k - s_k$, Startabstand $P_k = s_{k+1} - s_k$ und Peak-to-Peak-Jitter $J_P = \max(P_k) - \min(P_k)$.

1. Bestimmen Sie die ersten fünf A-Starts, Pulsbreite, Startabstände und J_P bei konstant 1 ms für B. Wiederholen Sie für konstant 3 ms für B. Ist die konstante Verlängerung allein schon Jitter? (2 P)
2. B benötigt abwechselnd 1, 3, 1, 3 ms. Bestimmen Sie die ersten fünf A-Starts, die vier Startabstände, Pulsbreite und J_P. (2 P)
3. B kehrt stattdessen nie zurück. Was geschieht mit weiteren A-/C-Aufrufen und mit weiterhin erlaubten Tickinterrupts? Warum ist `wake_at` setzen ohne Rückkehr kein Taskwechsel, und warum liefert eine Wake-Zeit keinen genauen Startzeitpunkt? (2 P)

Teil B: Vorbereiteter Programmvergleich · 25 Minuten

Aufbau kontrollieren · 5 Minuten

1. Die Dual-Core-Bare-Metal-Basis aus Vorlesung 1 ist bereits generiert, gebaut, geladen und korrekt gestartet. CM4 bezeichnet das Cortex-M4-Firmwareziel, CM7 das Cortex-M7-Ziel. Die generierte Bootfolge bleibt erhalten.
2. Prüfen Sie die tatsächlichen Takte und die fortschreitende **HAL-Zeitbasis mit 1 kHz** auf CM4. HAL bedeutet Hardware Abstraction Layer. `HAL_GetTickFreq()` meldet `HAL_TICK_FREQ_1KHZ`, `HAL_GetTick()` muss aber auch wirklich fortschreiten. Der Tickinterrupt wählt keinen anderen Schritt aus. Kein FreeRTOS aktivieren.
3. Entfernen Sie das alte Lab01-Modul aus dem CM4-Bauziel und dessen Timer-/LED-Aufrufe aus Übung 1, ohne die erforderliche Zeitbasisbehandlung zu löschen. Wählen Sie anhand Schaltplan und Boardbelegung einen freien herausgeführten GPIO-Ausgang, nur CM4 zugeordnet. GPIO heißt General-Purpose Input/Output. In CubeMX erhält er das **User Label** `LAB_MARKER`, Ausgangsmodus und Anfangspegel LOW. Port-/Pinmakros in CM4 `main.h` und den tatsächlichen Messanschluss prüfen. PA0 ist keine pauschale Vorgabe.
4. Werkzeugstände, Compiler-/Optimierungsoptionen und die CM4-/CM7-CMake-, Flash- und Debugkonfiguration dokumentieren. Die Quelle `lab02_demo.c` sowie deren Header sind bereits nur in CM4 eingebunden; erneute Generierung darf diese Einbindung nicht verlieren.

Im vorgesehenen Anwender-Include-Bereich von CM4 `main.c` steht `#include "lab02_demo.h"`. Nach erfolgreichem generierten HAL-/GPIO-Setup:

```
if (!Lab02_Init(LAB02_SHORT)) {
    Error_Handler();
}
```

Im Rumpf der **vorhandenen** `while (1)` steht:

```
Lab02_DispatchOnce();
```

Keine zusätzliche Endlosschleife, kein Step-Aufruf aus einem anderen Step und kein eigener PendSV- oder SysTick-Handler. Die vollständige Vorbereitung steht im Modul-README.

Vier Läufe lesen und vergleichen · 15 Minuten

Ändern Sie nur die Modusauswahl in `Lab02_Init`, bauen und starten Sie nach Reset neu. Die vorbereitete Basis und die Iterationszahlen bleiben gleich.

Auswahl	Was verändert wird
`LAB02_SHORT`	A, B und C bleiben mit kurzer Arbeit bereit
`LAB02_LONG_B`	B hat konstant mehr Iterationen
`LAB02_VARIABLE_B`	B wechselt zwischen kurzer und langer Arbeit
`LAB02_SLEEP_A`	A setzt nach Arbeit ein Wake-Ziel plus zehn Tickwerte und kehrt zurück

Die Standardzahlen 30000 beziehungsweise 120000 Iterationen sind **keine zugesicherten Millisekundenzeiten**. Diagnosefelder `lab02_diag.runs`, `result`, `idle_entries` und `failures` sind nur für einen bei angehaltenem CM4 konsistent gelesenen Zustand gedacht. Ein Debughalt gehört nicht in eine Zeitmessreihe.

Bearbeiten Sie dazu:

- Warum hat jeder ständig bereite Schritt eine Ringposition, aber keine fest zugesagte Zeitscheibe?
- Welcher Teil des Kontrollflusses verhindert einen A-Aufruf während laufender B-Arbeit, obwohl Tickinterrupts die Zeit fortschreiben?
- Was speichert der TCB, Task Control Block oder Verwaltungsdatensatz? Warum ist `running_idx` außerhalb eines Schrittaufrufs ungültig?
- Warum muss A nach erfolgreichem `Lab02_SleepAfterStep(10U)` selbst zurückkehren? Weshalb sind zehn Tickwerte keine genaue physische Startperiode?
- Warum prüft der Dispatcher höchstens drei Kandidaten? Was tut er bei drei Blocked-Einträgen? Lesen Sie dazu den Idle-Pfad mit `__WFI`, Wait For Interrupt; eine aktive Wake-Quelle ist erforderlich.
- Warum reicht `now >= wake_at` beim Überlauf nicht? Der Modulovergleich ist nur bei Warteintervallen und unbeobachteten Verspätungen strikt unter 2^{31} Millisekunden gültig.

Eine nie zurückkehrende B-Funktion wird **auf Papier** besprochen, nicht als verpflichtender Boardversuch ausgeführt. Zum Vergleich genügt die begrenzte längere Last.

Protokoll und Grenzen · 5 Minuten

Mit geeignetem Oszilloskop/Logikanalysator können beispielsweise 20 A-Pulse je verglichener Variante erfasst werden. Prüfen Sie den tatsächlichen Pinanschluss und gemeinsamen Massebezug. Die Pulsbreite umfasst A-Arbeit, Teile der GPIO-Aufrufe und mögliche Interruptunterbrechungen. Der Abstand steigender Flanken erfasst die Wiederaufrufe. Die ganze Dispatcherfunktion wird dadurch nicht markiert.

Konfiguration	Eigener Eintrag
Board / Firmwarepaket / Werkzeugversionen	
Compiler und Optimierung	

Konfiguration	Eigener Eintrag
M4- / M7-Takt, Tickquelle und Tickfrequenz	
Markerpin und Boardanschluss	
Iterationszahlen / Modus / Reset	
Trigger / Zeitauflösung / Anzahl	

Variante	Anzahl	Pulsbreite min./max.	Startabstand min./max.	Startab-standsspanne	Einheit / Status
Kurz					
konstant					
B lang					
konstant					
B wechselnd					
A wartet					
relativ					

Die Tabelle bleibt bis zur realen Messung leer. Status beispielsweise **gemessen**, **nur Code/Diagnose betrachtet**, **fehlgeschlagen** oder **nicht durchgeführt**. Ohne Messgerät werden keine Hardwarezeiten eingesetzt. Die Zahlen aus Aufgabe 5 gehören nicht in diese Spalten.

Abgabe und Quellen

Teil A: beschriftete Zeitachsen oder Intervalltabellen, Zwischenrechnungen und begründete Entscheidungen zu Aufgaben 1–5, insgesamt 32 Punkte. Teil B: .ioc-Konfiguration, Moduleinbindung, Werkzeug-/Buildangaben, Erklärung des Kontrollflusses und tatsächlicher Prüfstatus. Messergebnisse nur mit realem Protokoll und definierten Messpunkten abgeben.

Quellen: Vorlesungen 1–2; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*; Liu und Layland (1973) zur RMS-Schranke; die behandelte Antwortzeitanalyse; ST UM2408, RM0399 und PM0214 sowie die Dokumentation der tatsächlich verwendeten STM32-Erweiterung. Vollständiger Anwendungsbaustein und Integrationsanleitung.